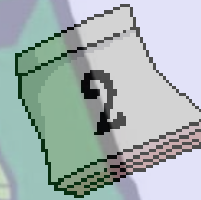




Chapitre



Les bases du langage



2- Les bases du langage

2.1 Les commentaires (1/4)

- **Tout programme (grand ou petit, simple ou complexe) contient (ou devrait contenir) des commentaires.**
- **Ils ont pour but d'expliquer :**
 - Ce qu'est sensé faire le programme,
 - Les conventions adoptées,
 - Tout autre information rendant le programme lisible à soi même et surtout à autrui.
- **Java dispose de trois types de commentaires :**
 - Les commentaires multilignes,
 - Les commentaires lignes,
 - Les commentaires de type documentation.

2- Les bases du langage

2.1 Les commentaires (2/4)



● Commentaires en lignes

- Les commentaires lignes débutent avec les symboles `//` et qui se terminent à la fin de la ligne.

`// Ce programme imprime la chaîne
// de caractères " bonjour " à l'écran`

`...`

- Ils sont utilisés pour des commentaires courts qui tiennent sur une ligne.

2- Les bases du langage

2.1 Les commentaires (3/4)



● Commentaires multilingnes :

- Un commentaire multiligne commence par les caractères ```/*` et se terminent par ```*/`.

`/* Ce programme imprime la chaîne
de caractères "bonjour" à l'écran
*/`

- A l'intérieur de ces délimiteurs toute suite de caractères est valide (sauf évidemment ```*/`).

2- Les bases du langage

2.1 Les commentaires (4/4)



● Commentaires de types documentation :

- Ces commentaires, appelés aussi commentaires **javadoc**, servent à documenter les classes que l'on définit.
- Ces commentaires sont encadrés entre `/**` et `*/`.

```
/** Documentation de la classe .  
*/
```

- Java exige que ces commentaires figurent avant la définition de la classe, d'un membre de la classe ou d'un constructeur.
- Ces commentaires serviront à produire automatiquement (avec l'outil **javadoc**) la documentation sous forme **HTML** à l'image de la documentation officielle de **Oracle**.

2- Les bases du langage

2.3 Les types de données élémentaires

- **Le langage **Java** est un langage fortement typé :**
 - Chaque variable et chaque expression possède un type bien défini.
 - Le choix d'un type est en fonction de l'étendu de valeur souhaitée pour vos variables.
- **Les types de données de **Java** sont divisés en deux grands groupes :**
 - Les types primitifs (ou de base)
 - Exemple : Les types numériques, le type booléen, Le type caractère.
 - Les types références.



2- Les bases du langage

2.4 Les types primitifs numériques (1/2)

POO : Java

Les types numériques entiers					
	byte	short	int		long
Taille en bits	8	16	32		64
Etendue	-128 .. 127	-32768 .. 32767	$-2^{31} .. 2^{31}-1$		$-2^{63} .. 2^{63}-1$



Le bit y est utilisé pour décrire les tailles.

- Un bit ne peut prendre que deux valeurs (0 ou 1).
- n** bits ne peuvent définir que **2^n** valeurs.
- Un **octet** ou **byte** en anglais = 8 bits.
- Un octet peut donc contenir **2^8** (soit 256) valeurs distinctes.



2- Les bases du langage

2.4 Les types primitifs numériques (2/2)

Les types numériques flottants

- De même que pour les types numériques, il existe deux types pour les nombres **flottants** :
 - float,
 - double.
- la seule différence résidant dans la taille utilisée pour stocker une valeur de ce type.

	float	double
Taille en bits	32	64
Exemple de valeur	3.25	3.25





2- Les bases du langage

2.5 Le type primitif booléen

- Ce type est introduit en Java par le mot clé **booléen**.
- Ce type est un vrai type **booléen**
 - il accepte seulement deux états :
 - l'un est nommé **true** :
 - Symbolise un état d'acceptation,
 - l'autre, nommé **false**,
 - Symbolise un état de réfutation.
- Attention, contrairement au langage **C**, le type booléen en **Java** n'est pas un sous-type numérique :
 - En **C** par exemple, la valeur 0 est considérée comme fausse et les autres valeurs entières comme vraies.

2- Les bases du langage

2.6 Le type primitif caractère

- Ce type, introduit par le mot clé **char**, et permet la gestion des caractères.
- **Java** utilise le codage de caractères universel **Unicode** qui est une extension du codage **ASCII**.
- Le codage **ASCII** utilise 8 bits et permet de représenter seulement 128 caractères.



2- Les bases du langage

2.7 Les types références

POO : **Java**

- Tout type non primitif, est un type de référence.
- Le rôle d'un type de référence est de référencer ou repérer une zone mémoire.
- Un type de référence a un rôle analogue à celui des pointeurs du langage **C**.

Plus de détails plus loin ...



2- Les bases du langage

2.8 Déclaration et initialisation des variables

Syntaxe :

 **Type** identificateur [= constante ou expression];

 `int` NbredeMois = 12 ; `int` NbredeMois = 4*3 ;

 `int` nbrDeDoigts = 012 ;

 `boolean` Unboolean = false ;

 `float` Unfloatant = 1.3f ;

 `char` Uncaractère = 'c' ; `char` Uncaractère = '\n' ;

 `String` Unstring= " bonjour \n " ;

 Et éventuellement, un « **modificateur d'accès** ou **de visibilité** » : `final float` pi=3.14159



2- Les bases du langage

2.9- Les opérateurs



● Les opérateurs dans **Java** sont regroupés par :

- type d'opérations :
 - d'affectation
 - numérique,
 - de comparaison,
 - logique,
 - sur les chaînes de caractères,
 - de manipulations binaires.
- le nombre d'opérandes :
 - unaire,
 - binaire,
 - ternaire.



2- Les bases du langage

2.10- Opérateurs unaires

Opérateurs unaires	Action	Exemple
-	négation	$i = -j$
++	incrément de 1	$i = j++$ ou $i = ++j$
--	décrément de 1	$i = j--$ ou $i = --j$

● ++ et -- peuvent **préfixer** ou **postfixer** la variable.

● $i = j++$: post-incrément

- La valeur en cours de j est affectée à i et ensuite la valeur de j est incrémentée de 1.

● $i = ++j$: pré-incrément

- La valeur en cours de j est incrémentée de 1 et ensuite la valeur de j est affectée à i .

2- Les bases du langage

2.11- Opérateurs binaires

POO : Java



T. CHAARI

Opérateurs binaires	Action	Exemple	Syntaxe équivalente
+	addition	<code>i = j+k;</code>	
<code>+=</code>		<code>i += 2;</code>	<code>i = i + 2</code>
-	soustraction	<code>i = j - k;</code>	
<code>-=</code>		<code>i -= j;</code>	<code>i = i - j</code>
*	multiplication	<code>x=2*y;</code>	
<code>*=</code>		<code>x *=x;</code>	<code>x = x * x</code>
/	division (tronque si les arguments sont entiers)	<code>i =j/k;</code>	
<code>/=</code>		<code>x /= 10;</code>	<code>x = x /10</code>
%	modulo	<code>i = j %k;</code>	
<code>%=</code>		<code>i %=2</code>	<code>i = i %2</code>



2- Les bases du langage

2.12- Opérateurs relationnels

- Le résultat d'une comparaison est une valeur booléenne (**vrai** ou **faux**).
- Dans le langage **Java**, le résultat d'une comparaison est **true** ou **false**.

Opérateurs relationnels	Action	Exemple
<	plus petit que	x<i;
>	plus grand que	i>100;
<=	plus petit ou égal que	j<=k;
>=	plus grand ou égal que	c>='a';
==	égal à	i==20;
!=	différent de	c!= 'z';

2- Les bases du langage

2.13- Opérateurs logiques

POO : **Java**



T. CHAARI

Opérateurs logiques		Action	Exemple	Syntaxe équivalent
!		négation	! p;	
&		ET	p & (i<10)	
		OU	p q	



2- Les bases du langage

2.14- Opérateurs ternaire

- Un unique opérateur **ternaire**.
- Cette expression est une sorte de **si-alors-sinon** sous forme d'expression :
 - $a = (\text{condition } e) ? x : y$
 - si la condition **e** est **vraie** alors **a** vaut **x** sinon elle vaut **y**.
 - Exemple : $a = (v == 2) ? 1 : 0;$
 - Cette expression affecte à la variable **a** la valeur **1** si **v** vaut **2**, sinon affecte à la variable **a** la valeur **0**.



2- Les bases du langage

2.15 structures de contrôle : if

● Syntaxe :

```
if (expression booléenne) instruction ;
```

● Exemple :

```
public class IfApplication1 {
    public static void main(String[ ] args)
    {
        int i = 4 ;
        if ( i %2 != 0 ) System.out.print ( " i est impair " );
    }
}
```

Résultat d'affichage



Attention : L'expression logique attendue est obligatoirement de type **boolean**.

- si l'on écrit `if ( i = 1 )`, le compilateur détectera une erreur, car le type de l'expression logique est alors `int`.



2- Les bases du langage

2.15 structures de contrôle : if - else

Syntaxe	Exemple
<pre> if (expression booléenne) { instruction 1 ; instruction i ; } else { instruction j ; instruction n ; } </pre>	<pre> public class If_elseApplication { public static void main(String[] args) { int i = 4 ; if (i % 2 == 0) { System.out.println (" i est pair "); } else { System.out.println (" i est impair "); } } } </pre>
Résultat d'affichage i est pair	



2- Les bases du langage

2.15 structures de contrôle : if – else - if

Syntaxe	Exemple
<pre> if (expression booléene 1) { instruction 1 ; instruction i ; } else if (expression booléene 2) { instruction j ; instruction m ; } else { instruction m+1 ; } </pre>	<pre> public class if_else_ifApplication { public static void main(String[] args) { int i = 1, j =2 ; if (i == j) { System.out.println (" i est égal à j "); } else if (i>j) { System.out.println (" i est supérieur à j"); } else { System.out.println (" i est positive "); } } } </pre>
Résultat d'affichage i est positive	



2- Les bases du langage

2.15 structures de contrôle : switch

Syntaxe	Exemple
<pre> switch (variable) { case valeur 1 : instr1_1; instr1_2; ... break; ... case valeur N : instrN_1; instrN_2; break; default: /* optionnel */ instrD_1; instrD_2; ... break; } </pre>	<pre> public class SwitchApplication { public static void main(String[] args) { int i = 1, switch (i) { case 0 : System.out.println (" 0 "); break; case 1 : System.out.println (" 1 "); break; case 2 : System.out.println (" 2 "); break; default : System.out.println (i); break; } } } </pre>
Résultat d'affichage 1	



2- Les bases du langage

2.15 structures de contrôle : for

Exemple :

```
public class ForApplication {  
    public static void main(String[ ] args)  
    {  
        int somme = 0;  
        for ( int compteur = 1, max = 4 ; compteur < max ; compteur++)  
        {  
            somme = somme + compteur;  
        }  
        System.out.println (somme );  
    }  
}
```

Résultat d'affichage

6

Fonctionnement :

- initialisation du compteur,
- comparaison avec max,
- réalisation des instructions,
- Incrémentation du compteur et on recommence.

2- Les bases du langage

2.15 structures de contrôle : while

● Exemple :

```
public class WhileApplication {  
    public static void main(String[ ] args)  
    {  
        int compteur =1; int max = 4 ; int somme = 0;  
  
        while (compteur < max )  
        {  
            somme = somme + compteur ;  
            compteur++;  
        }  
  
        System.out.println (somme );  
    }  
}
```

Résultat d'affichage

6





2- Les bases du langage

2.15 structures de contrôle : do - while

● Exemple :

```
public class Do_WhileApplication {
    public static void main(String[ ] args)
    {
        int compteur =1; int max = 4 ; int somme = 0;

        do
        {
            somme = somme + compteur ;
            compteur++;
        }
        while (compteur < max ) ;

        System.out.println (somme );
    }
}
```

Résultat d'affichage

6



2- Les bases du langage

2.15 structures de contrôle : **break**

Utilisation :

- Pour sortir d'une structure de boucle avant que la condition du test soit remplie.
- Quand la boucle rencontre une instruction **break**, elle se termine immédiatement en ignorant le code restant.

```
public class BreakApplication {
    public static void main(String[ ] args)
    {
        int compteur =0;
        while (compteur < 10 )
        {
            System.out.println (compteur );
            compteur++;
            if (compteur == 5) break;
        }
    }
}
```

Résultat d'affichage

0
1
2
3
4



2- Les bases du langage

2.15 structures de contrôle : continue



Utilisation :

- Pour ignorer le reste de la boucle et reprendre l'exécution à l'itération suivante de la boucle.

```
public class ContinueApplication {
    public static void main(String[ ] args)
    {
        int compteur =0;
        for(compteur=0 ; compteur< 6 ; compteur++ )
        {
            if (compteur == 4) continue;
            System.out.println (compteur );
        }
    }
}
```

Résultat d'affichage

0
1
2
3
5

2- Les bases du langage

2.16 Les conversions des types (1/8)

● Il y a 2 catégories de conversions possibles :

● Conversions explicites :

- celles faites sur une demande explicite par un programmeur.

● Conversions implicites :

- celles faites automatiquement par un compilateur :

- lors d'une affectation,
- lors d'une promotion arithmétique,
- lors d'un passage de paramètres (lors de l'invocation d'une méthode),





2- Les bases du langage

2.16 Les conversions des types (2/8)



Conversion explicite :



Objectif :

- changer le type d'une donnée si besoin.



Comment ? :

- Préfixer l'opérande par le type choisi.
- Encadrer le type choisi par des parenthèses.



Exemple :

● `double d = 2.5 ;`

● `long l = (long)d ;`

2- Les bases du langage

2.16 Les conversions des types (3/8)

● Conversion implicite lors d'une affectation :

● Objectif :

- changer automatiquement le type d'une donnée si besoin.

● Comment ? :

● Exemple :

- Un type **char** peut être utilisé partout où une valeur de type **int** est permise.



2- Les bases du langage

2.16 Les conversions des types (4/8)

● Conversion implicite lors d'une affectation :

● Illustration pour l'exemple 1:

```
public class ProgrammeAffectation {
    public static void main(String[ ] args)
    {
        int i;
        short j = 2;
        i = j;
        float k = 1.2 f;
        i = k;
    }
}
```

Conversion implicite

Erreur de compilation

● Solution :

```
i = (int) k;
```


2- Les bases du langage

2.16 Les conversions des types (5/8)

● Conversion implicite lors d'une affectation :

- Illustration pour l'exemple 2:

```
public class ProgrammeSoustraction {  
    public static void main(String[ ] args)  
    {  
        int i;  
        i = 'A';  
        System.out.print( i ) ;  
    }  
}
```

Conversion implicite

- Résultat d'affichage :

65



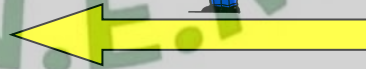
2- Les bases du langage

2.16 Les conversions des types (6/8)

● Conversion implicite lors d'une promotion arithmétique :

- Objectif :
 - Si un opérateur s'applique sur deux arguments de type différent un des deux arguments sera converti dans le type de l'autre.
- Exemple :

```
public class ProgrammeSoustraction {
    public static void main(String[ ] args)
    {
        int i;
        i = 'A'-1;
        System.out.print( i );
    }
}
```



Conversion implicite

● Résultat d'affichage :

64



2- Les bases du langage

2.16 Les conversions des types (7/8)

- Conversion implicite lors d'un passage de paramètres (lors de l'invocation d'une méthode) :

```
public class ProgrammeSoustraction {
```

```
    static short entierShort;
```

```
    static void Afficher( int entier )  
    {  
        System.out.print( entier ) ;  
    }
```

```
    public static void main(String[ ] args)  
    {  
        entierShort = 2 ;  
        Afficher(entierShort) ;  
    }  
}
```



Invocation de la
méthode Afficher



2- Les bases du langage

2.16 Les conversions des types (8/8)



Attention :

- Il n'y a pas de conversion possible (implicite ou explicite) entre un type **entier** et le type **boolean** :

```
int i = 0;

if (i)
{
    .....
}

if (i != 0)
{
    .....
}
```

// Erreur à la compilation

// Ok





2- Les bases du langage

2.17 Les tableaux (1/2)

● Déclaration

- monodimensionnel
 - `int[] tableau_Entier;`
 - équivalent à : `int tableau_Entier[];`
- multidimensionnel
 - `Color rgb_cube[][][];`

● Création et initialisation

- `int[] primes = {1, 2, 3, 5, 7};`
- `int[] tableau_Entier = new int[50];`
- `tableau_Entier[0] = 3;`
- `rgb_cube = new Color[256][256][256];`

2- Les bases du langage

2.17 Les tableaux (2/2)



- Les indices des tableaux commencent par **0**.
- Exemple d'utilisation des tableaux :

```
public class TableauApplication {  
  
    public static void main(String[ ] args)  
    {  
        int[ ] tableau_Entier = new int[50]; // création  
        System.out.println(tableau_Entier.length);  
        tableau_Entier[51]=10;  
    }  
}
```

Résultat d'affichage

```
50  
java.lang.ArrayIndexOutOfBoundsException: 51
```